

```
python
```

```
(self, creationid, creationseq):  
    'm a new Gene object!')  
    y constructor got a param: " + str(creationid))  
    ssigning that param to my id instance variable.  
    = creationid  
    imilarly, assigning to my sequence instance var:  
    uence = creationseq
```

```
(self):  
    y id is: " + str(self.id))
```

```
n(self):  
    y sequence len is: " + str(len(self.sequence)))
```

```
reating geneA:")  
342", "CATTGAC")
```

```
reating geneB:")  
4B", "TTACTAGA")
```

```
sking geneA to print_id():")
```

```
sking geneB to print_id():")
```

```
sking geneA to print_len():")  
)
```



Object Oriented Programming in Python

Most programs and techniques used so far follow a procedural approach. However, for large and complex projects with large teams, the object-oriented approach is invaluable. Python, being a versatile tool, allows for both object-oriented and procedural programming.



by Prof. M. Iqbal Bhat



Procedural vs Object-Oriented

1 Data and Code Separation

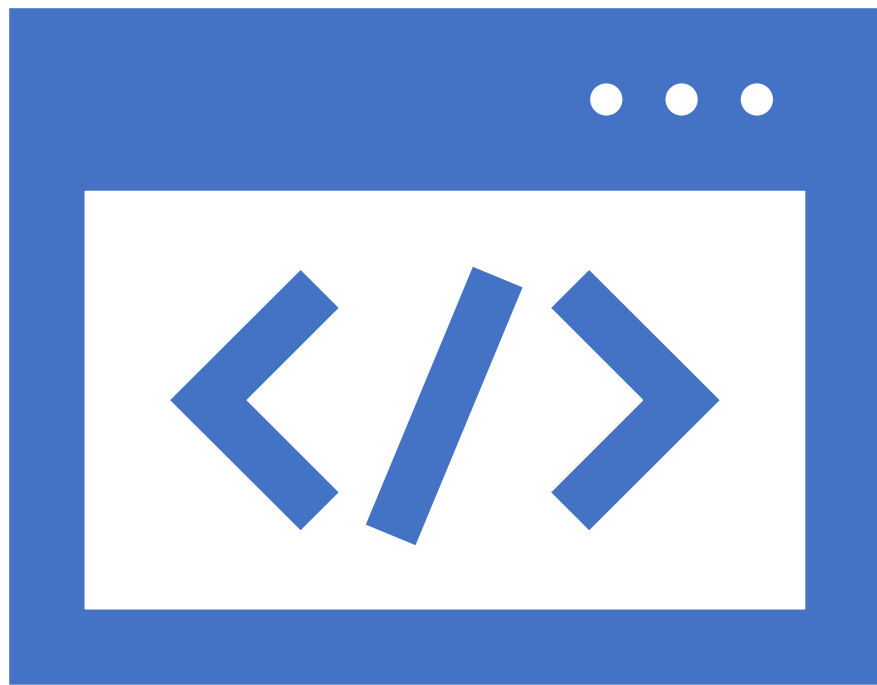
In procedural programming, there are separate worlds for data and code. Functions can use data, but not vice versa. However, special types of data called methods can use functions.

2 Data and Code Combination

The object-oriented approach combines data and code in classes. Each class acts as a recipe to create objects with properties and methods. Classes can be modified or new ones can be created. Objects interact with each other and protect sensitive data.

3 Hierarchies and Categories

Class hierarchies, like categories, enable better organization. For instance, vehicles have the ability to move, distinguishing them from other beings. Vehicles can have subclasses like land, water, air, and space vehicles. Land vehicles can further be divided into wheeled, tracked, and hovercrafts.



Python is a universal tool for both object and procedural programming.

OOP in Python:



The **object approach** suggests a completely different way of thinking. The data and the code are enclosed together in the same world, divided into classes.



Every **class** is like a **recipe** which can be used **when you want to create a useful object**. You may produce as many objects as you need to solve your problem.



Every object has a set of traits (they are called properties or attributes – we'll use both words synonymously) and is able to perform a set of activities (which are called methods).

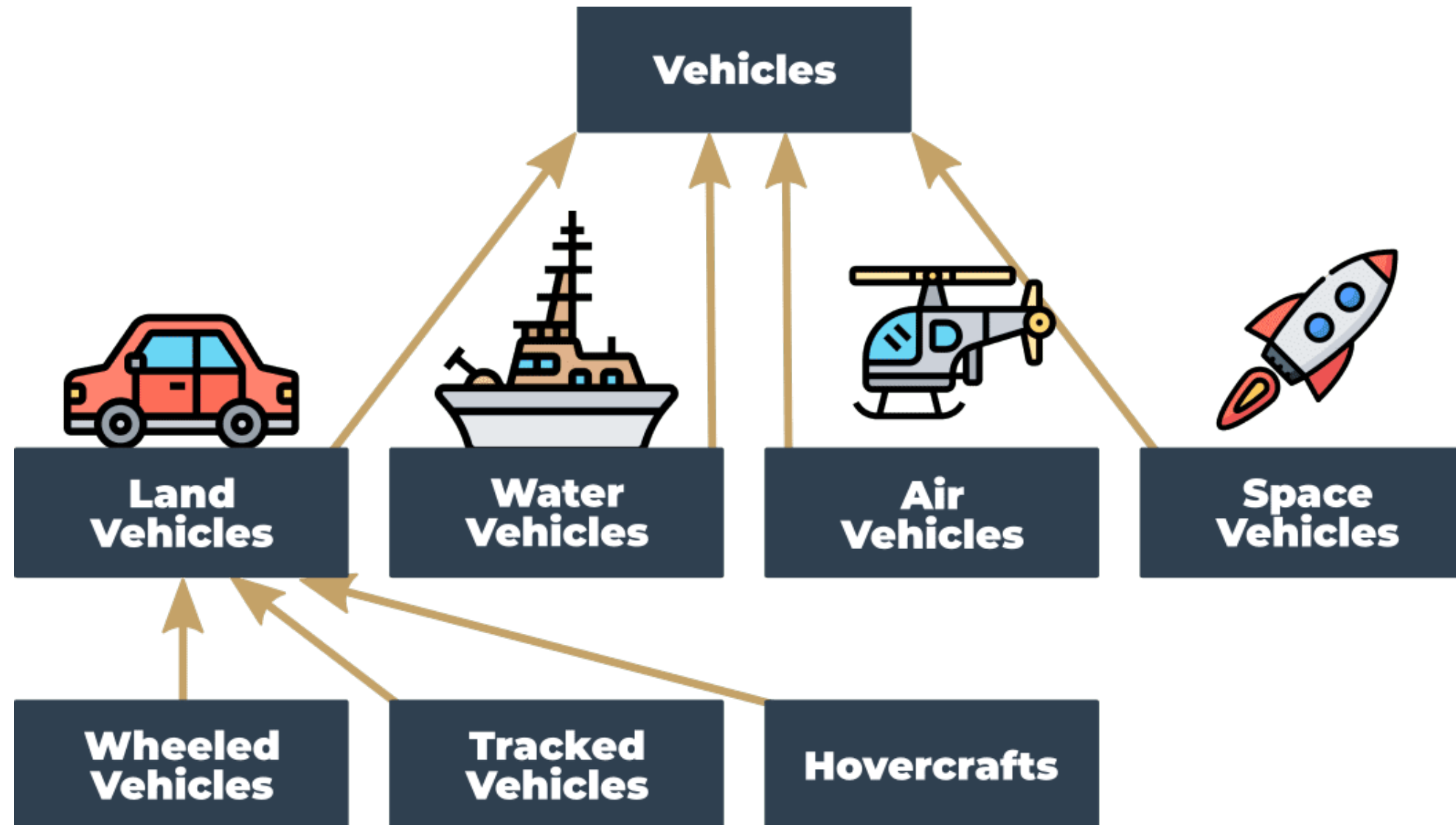


The recipes may be modified if they are inadequate for specific purposes and, in effect, new classes may be created. These new classes inherit properties and methods from the originals, and usually add some new ones, creating new, more specific tools.

Class Hierarchy



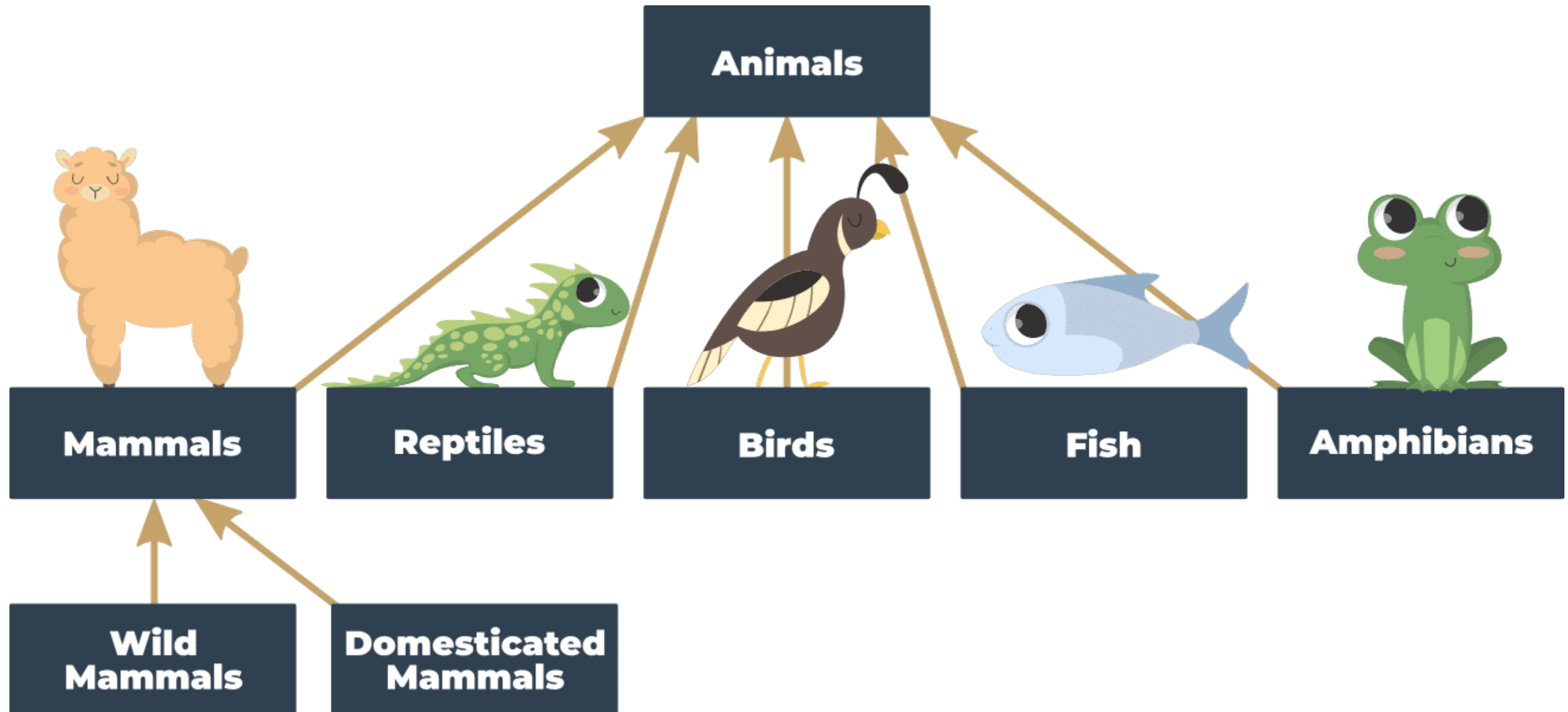
The *vehicles* class is very broad. Too broad. We have to define some more **specialized classes**, then. The specialized classes are the **subclasses**. The *vehicles* class will be a **superclass** for them all.



The hierarchy grows from top to bottom, like tree roots, not branches.

Animal Kingdom Hierarchy

The animal kingdom hierarchy showcases subclasses like mammals, reptiles, birds, fish, and amphibians. Mammals can be further categorized into wild and domesticated.



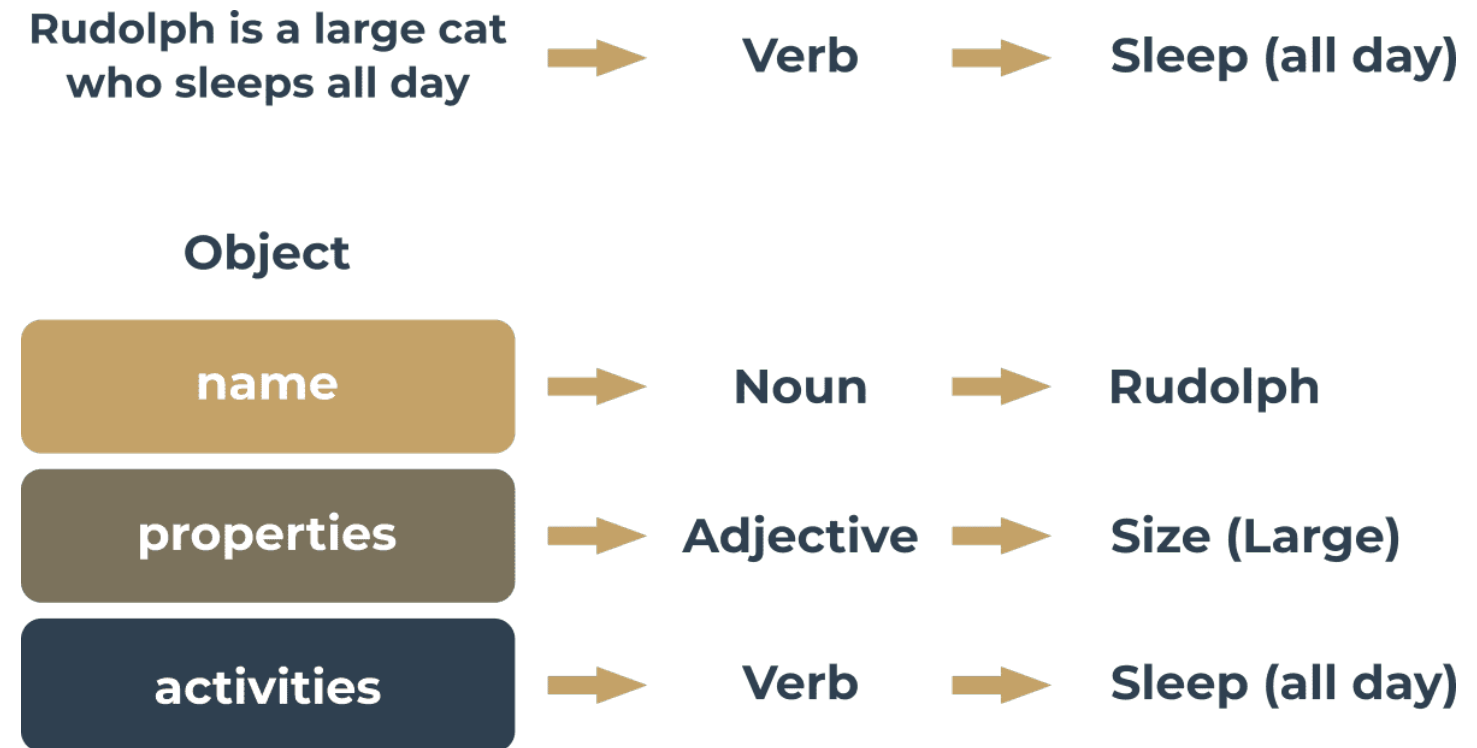
What is an object?

- A class (among other definitions) is a **set of objects**. An object is a **being belonging to a class**.
- An object is an **incarnation of the requirements, traits, and qualities assigned to a specific class**. This may sound simple, but note the following important circumstances. Classes form a hierarchy.



What does an object have?

- The object programming convention assumes that every existing object may be equipped with three groups of attributes:
 - an object has a name that uniquely identifies it within its home namespace (although there may be some anonymous objects, too)
 - an object has a set of individual properties which make it original, unique, or outstanding (although it's possible that some objects may have no properties at all)
 - an object has a set of abilities to perform specific activities, able to change the object itself, or some of the other objects.



OOP Programming in Python:

- Object programming is the art of defining and expanding classes.
- A class is a model of a very specific part of reality, reflecting properties and activities found in the real world.
- The class you define has nothing to do with the object: the existence of a class does not mean that any of the compatible objects will automatically be created. The class itself isn't able to create an object – you have to create it yourself, and Python allows you to do this.
- First Class:

```
1  class TheSimplestClass:  
2      pass
```

The definition begins with the keyword class. The keyword is followed by an **identifier which will name the class** (note: don't confuse it with the object's name – these are two different things).

Next, you add a **colon (:)**, as classes, like functions, form their own nested block. The content inside the block define all the class's properties and activities.

The pass keyword fills the class with nothing. It doesn't contain any methods or properties.

OOP Programming in Python:

- The newly defined class becomes a tool that is able to create new objects. The tool has to be used explicitly, on demand.
- Imagine that you want to create one (exactly one) object of the TheSimplestClass class.
- To do this, you need to assign a variable to store the newly created object of that class, and create an object at the same time.
- You do it in the following way:

```
1 | my_first_object = TheSimplestClass()
```

The act of creating an object of the selected class is also called an instantiation (as the object becomes an instance of the class).

OOP Programming in Python:

- The `__init__()` method in Python is a special method that is automatically called when an object is created from a class.
- It stands for "initialize" and is often referred to as the constructor of a class.
- The primary purpose of the `__init__()` method is to initialize the attributes of the object.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Creating an instance of the Dog class
my_dog = Dog(name="Buddy", age=3)

# Accessing attributes of the object
print(f"{my_dog.name} is {my_dog.age} years old.")
```

Self Parameter: The first parameter of the `__init__()` method is always `self`. This parameter represents the instance of the class itself. It is a convention in Python to use `self` as the name of the first parameter, but it is not a keyword; you could technically use any name.

Procedural to Object Oriented Approach

What is a stack?

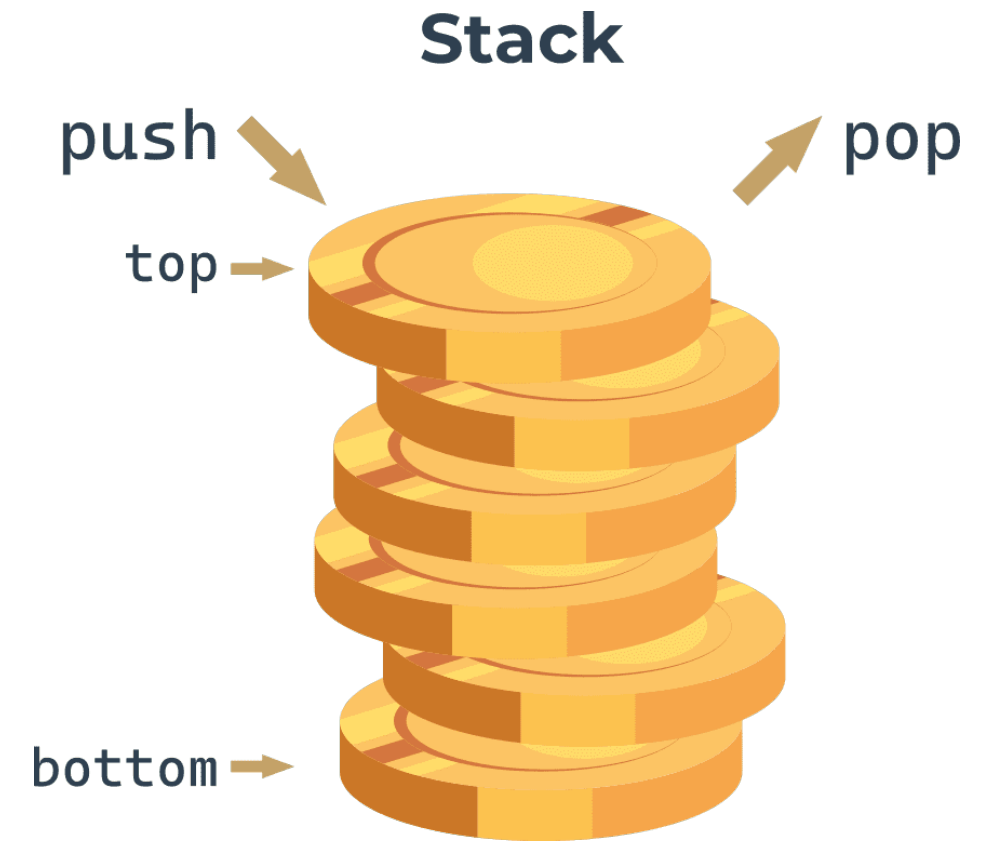
A stack is a structure developed to store data in a very specific way.



The alternative name for a stack (but only in IT terminology) is **LIFO**(Last In – First Out). The coin that came last onto the stack will leave first.

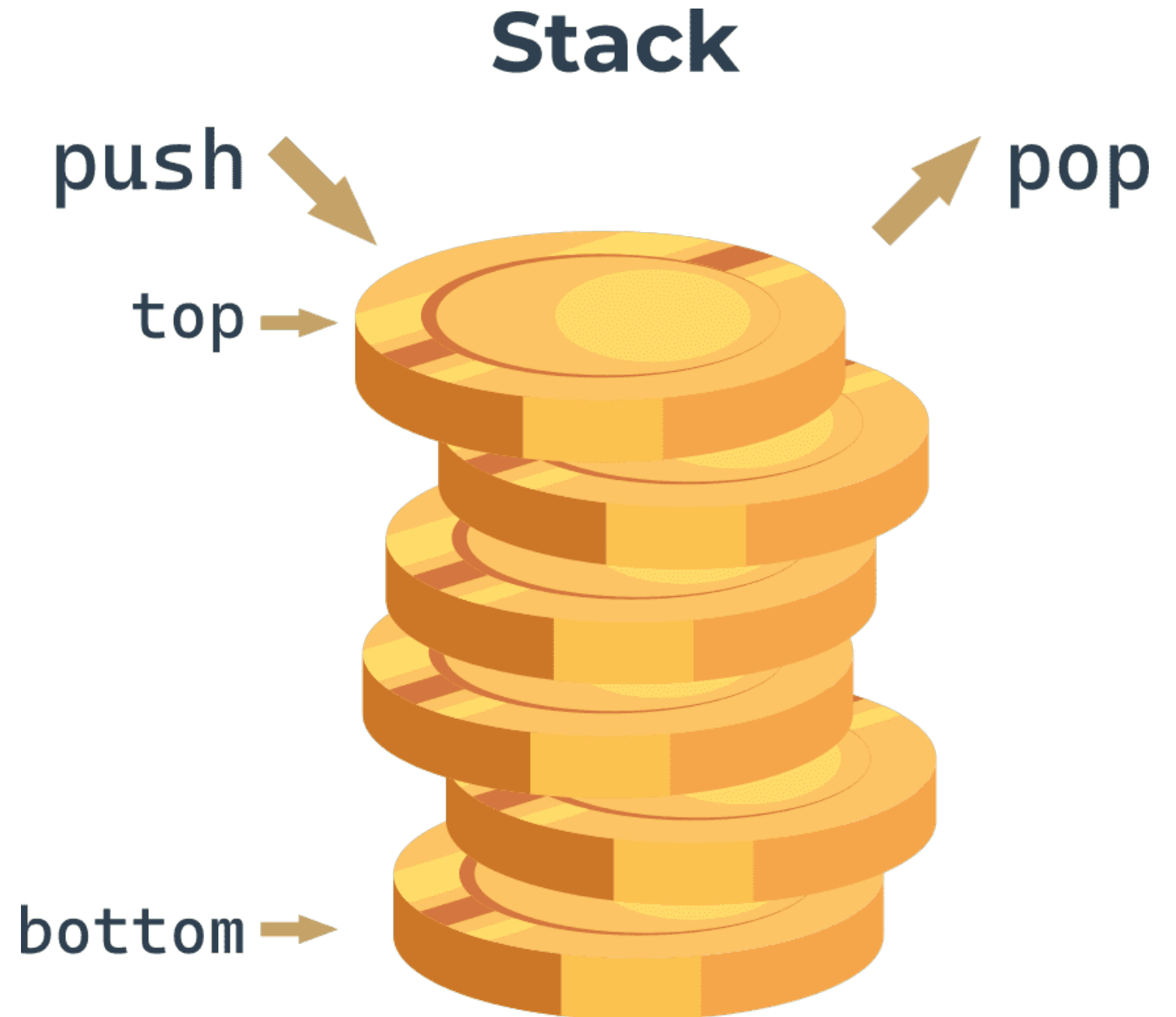


A stack is an object with two elementary operations, conventionally named push (when a new element is put on the top) and pop (when an existing element is taken away from the top).

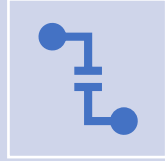


Stack Implementation in Procedural Programming

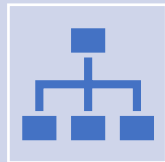
```
1 stack = []  
2 |  
3 def push(val):  
4     stack.append(val)  
5  
6 def pop():  
7     val = stack[-1]  
8     del stack[-1]  
9     return val  
10 push(3)  
11 push(2)  
12 push(1)  
13  
14 print(pop())  
15 print(pop())  
16 print(pop())
```



Drawbacks in Procedural Stack Implementation:



The essential variable (the **stack** list) is highly vulnerable; anyone can modify it in an uncontrollable way, destroying the stack, in effect; this doesn't mean that it's been done maliciously – on the contrary, it may happen as a result of carelessness, e.g., when somebody confuses variable names; imagine that you have accidentally written something like this: `stack[0] = 0`



the functioning of the stack will be completely disorganized;



it may also happen that one day you need more than one stack; you'll have to create another list for the stack's storage, and probably other push and pop functions too;



it may also happen that you need not only push and pop functions, but also some other conveniences; you could certainly implement them, but try to imagine what would happen if you had dozens of separately implemented stacks.

Solution by OOP:

01

The ability to hide (protect) selected values against unauthorized access is called encapsulation; the encapsulated values can be neither accessed nor modified if you want to use them exclusively;

02

When you have a class implementing all the needed stack behaviors, you can produce as many stacks as you want; you needn't copy or replicate any part of the code;

03

The ability to enrich the stack with new functions comes from inheritance; you can create a new class (a subclass) which inherits all the existing traits from the superclass, and adds some new ones.

Stack Implementation in OOP:

```
1 class Stack:
2     def __init__(self):
3         self.__stack_list = []
4     def push(self, val):
5         self.__stack_list.append(val)
6     def pop(self):
7         val = self.__stack_list[-1]
8         del self.__stack_list[-1]
9         return val
10 stack_object = Stack()
11 stack_object.push(3)
12 stack_object.push(2)
13 stack_object.push(1)
14
15 print(stack_object.pop())
16 print(stack_object.pop())
17 print(stack_object.pop())
```


Python's Versatility



Object-oriented Python

Python is a powerful tool for object-oriented programming, allowing the creation of classes and objects with ease.

```
sample <- rnorm(count, mean = mean, sd = sd)
looped <- 0

while(any(sample < lower | sample > upper)) {
  # fix the sample
  bad <- sample < lower | sample > upper # logical
  bad_values <- sample[bad]
  count_bad <- length(bad_values)

  new_vals <- rnorm(count_bad, mean = mean, sd = sd)
  sample[bad] <- new_vals

  if(looped > 100000) {
    return(NA)
  }
  looped <- looped + 1
}
return(sample)

sample_trunc <- rnorm_trunc(0, 30, 1000, mean = 20, sd = 10)
if(!is.na(sample_trunc)) {
  hist(sample_trunc)
}
```

Procedural Python

Python also excels in procedural programming, where functions operate on separate data entities.



Large-scale Projects

For complex projects with large teams, Python's versatility facilitates efficient collaboration in both object-oriented and procedural programming paradigms.

Benefits of Object-Oriented Programming

Modularity

Object-oriented programming encourages modularity, allowing for the division of a complex system into smaller, manageable components.

Code Reusability

By utilizing classes and objects, code reusability becomes a prominent advantage, reducing redundancy and improving development speed.

Data Encapsulation

Object-oriented programming encapsulates data within objects, protecting sensitive information and ensuring secure interactions.

Team Collaboration

The object-oriented approach accommodates effective collaboration among team members, simplifying project management and maintenance.

Procedural Flexibility in Python

1 Scripting Capabilities

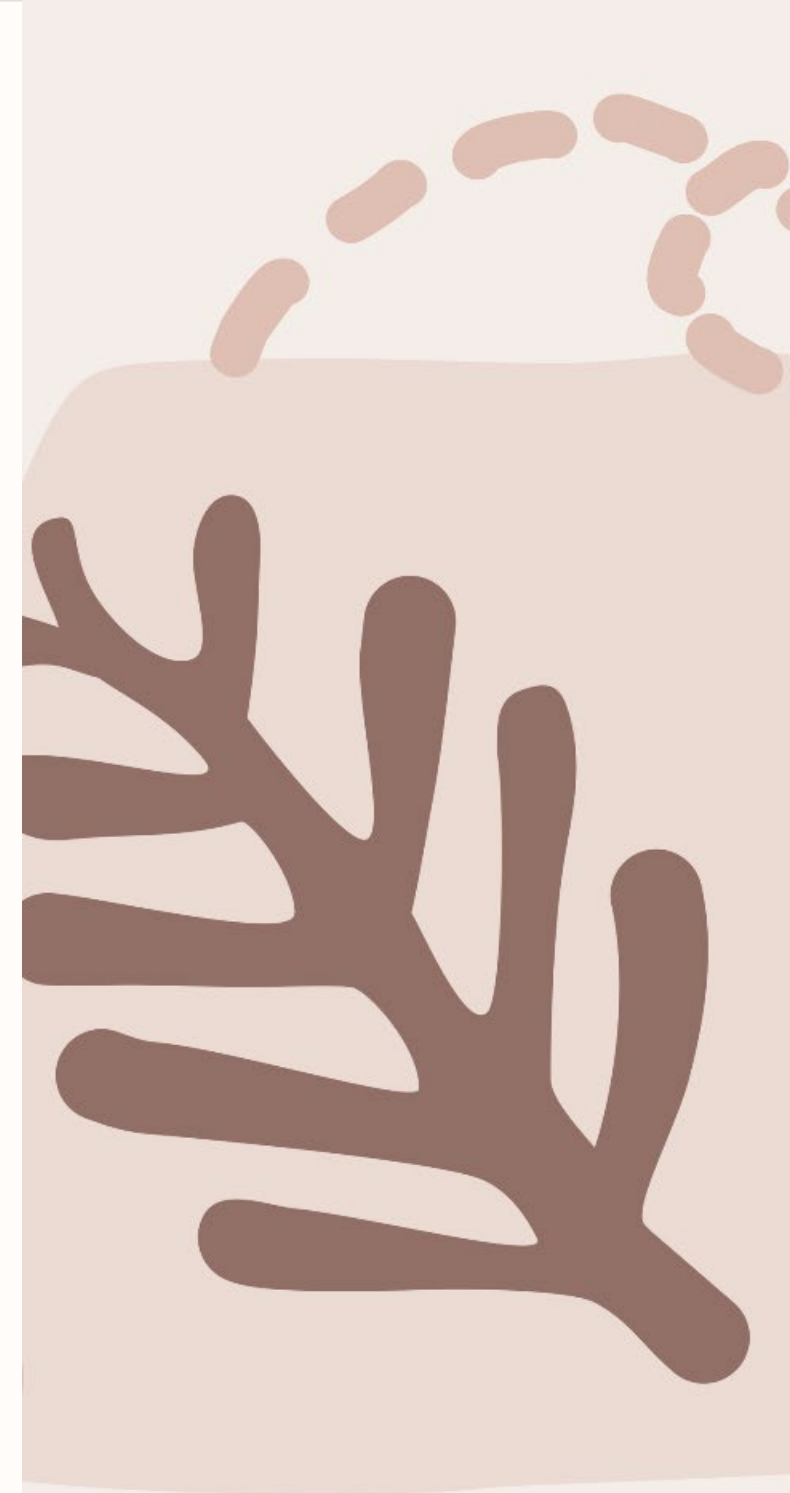
Python's ability to serve as a powerful scripting language enhances its procedural programming capabilities.

2 Procedural Libraries

A wide range of procedural libraries in Python provides developers with vast functionality to leverage in their projects.

3 Code Organization

Procedural programming in Python offers developers the flexibility to organize their code in a logical and structured manner.



Choosing the Right Approach



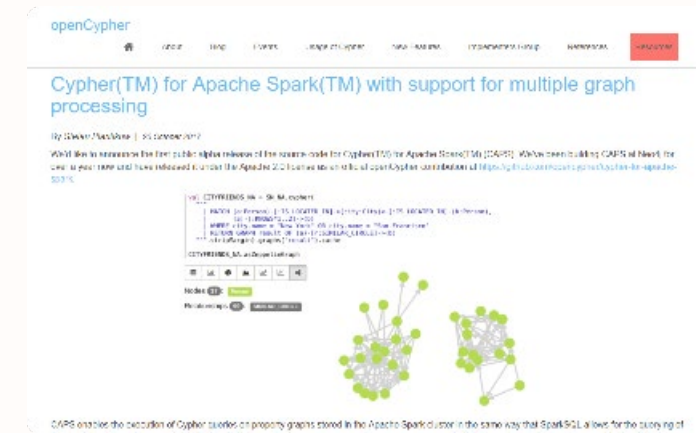
Complex Projects

For large and complex projects with extensive functionality, the object-oriented approach is highly recommended to ensure better organization and maintainability.



Team Collaboration

If multiple developers are involved in a project, the object-oriented approach enables smoother collaboration and code integration.



Procedural Scripting

For script development and smaller projects with predefined sequences, procedural programming in Python can be a suitable choice.



Conclusion

Understanding the benefits and differences between procedural and object-oriented programming is crucial for developers. Python's versatility allows for seamless transition and adoption of either approach based on project requirements and complexity.